

# cc-agent-java 架构：从框架、组件、进程/线程三个维度理解

用 C++ 工程师的思维方式来理解这个 Java Spring Boot 项目。

## 一、框架维度：三层基础设施



### 1.1 JVM — 运行时环境

**是什么：** 一个程序 (java.exe) ，负责执行编译后的 .class 文件。

**C++ 类比：** 没有直接对应。C++ 编译后是机器码，OS 直接执行。Java 编译后是字节码，必须由 JVM 解释/编译执行。

**在这个项目里：** JVM 是操作系统看到的唯一进程。Tomcat、Spring 都是 JVM 内部的线程/对象。

## 1.2 Spring Boot — 对象管理框架

**是什么：** 一个 Java 库（一组 jar 包），核心功能是**自动创建对象并管理对象之间的依赖关系**。

**C++ 类比：** C++ 没有这种框架。你通常手动 `new` 对象、手动管理生命周期、手动传依赖。Spring Boot 帮你做这些事。

**在这个项目里做了什么：**

Spring Boot 启动时（`SpringApplication.run()` 被调用后）：

1. 扫描 `com.example.ccagent` 包下所有类
2. 找到标记了 `@Component/@Service/@RestController` 的类
3. 为每个类创建一个实例（调用构造函数）
4. 把实例存到一个"容器"里（本质是一个大 Map）
5. 检查每个实例的字段，如果标记了 `@Autowired`，从容器里找到匹配的实例，赋值给它
6. 启动 Tomcat，注册 HTTP 路由
7. 开始监听 8080 端口

**代码体现：**

```
// 你只需要标记，不需要手动创建
@Component
public class FileReadTool implements AgentTool { ... }

@Service
public class AgentService {
    @Autowired
    private AnthropicClient anthropicClient; // Spring 自动赋值
}
```

## 1.3 Tomcat — Web 服务器

**是什么：** 一个 Java 写的 HTTP 服务器，被 Spring Boot 内嵌在应用里。

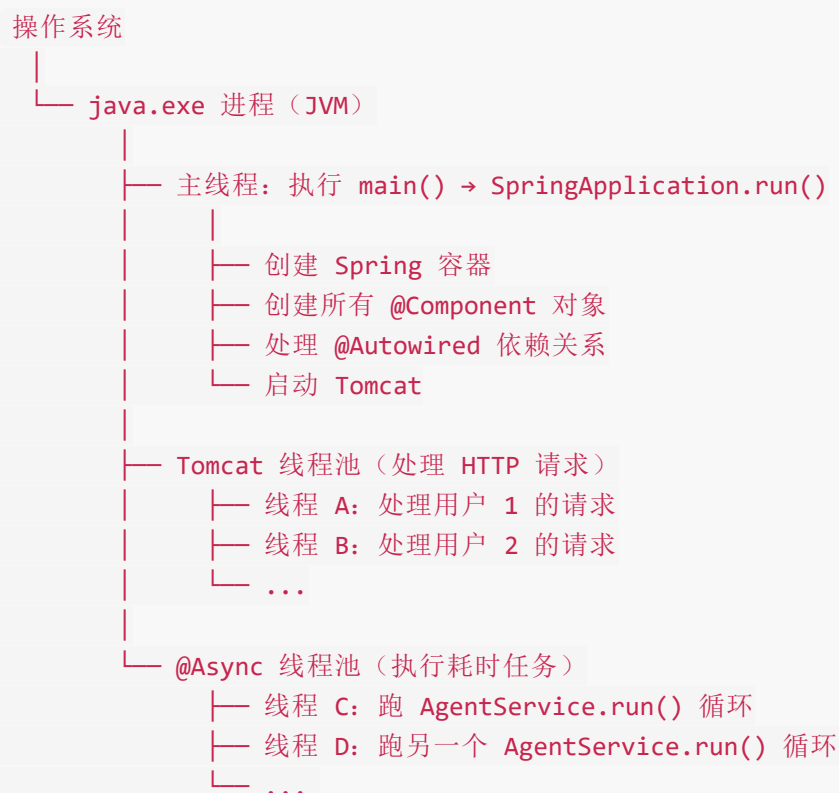
**C++ 类比：** 类似你在 C++ 里用 libevent/Boost.Asio 写的 TCP 服务器，监听某个端口，接收 HTTP 请求。

**在这个项目里做了什么：**

1. 监听 8080 端口
2. 收到 HTTP 请求后，交给 Spring 处理
3. Spring 匹配 URL → 调用对应的方法
4. 方法返回结果 → Tomcat 把结果发回客户端

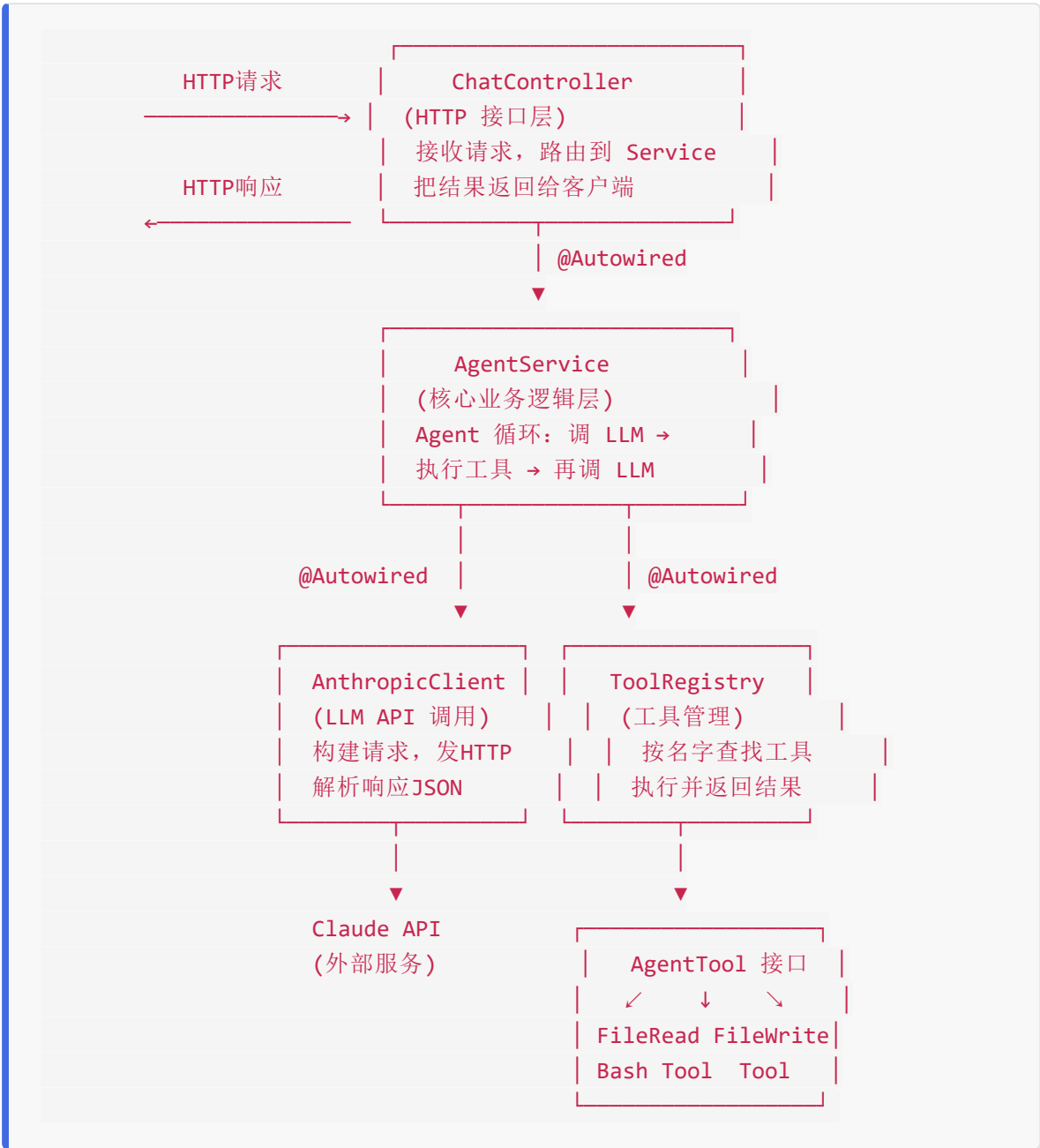
**关键点：** Tomcat 在 JVM 内部运行，不是一个独立进程。它是 JVM 内部的多个线程。

## 1.4 三者之间的关系



## 二、组件维度：每个类的职责和关系

### 2.1 组件总览



## 2.2 各组件详解

### ChatController (HTTP 接口层)

| 属性     | 值                               |
|--------|---------------------------------|
| 类标记    | @RestController                 |
| 生命周期   | 单例（Spring 创建一个实例，所有请求共用）        |
| 依赖     | AgentService （通过 @Autowired 注入） |
| 线程模型   | 方法在 Tomcat 线程上执行                |
| C++ 类比 | RPC Server 的 handler 注册代码       |

**做什么：** 1. 接收 HTTP 请求（POST /api/chat） 2. 从请求体提取用户消息 3. 调用 `agentService.run(message)` 4. 把结果返回给客户端

**不做什么：** - 不调 Claude API - 不执行工具 - 不管理对话历史

**AgentService（核心业务逻辑）**

| 属性     | 值                                     |
|--------|---------------------------------------|
| 类标记    | @Service                              |
| 生命周期   | 单例                                    |
| 依赖     | AnthropicClient 、 ToolRegistry        |
| 线程模型   | @Async 使 <code>run()</code> 方法在独立线程执行 |
| C++ 类比 | RPC 请求的 handler 实现                    |

**做什么：** 1. 管理对话历史（ `List<Message> history` ） 2. 运行 Agent 循环（for 循环，最多 10 轮） 3. 每轮：调 LLM → 检查是否有工具调用 → 执行工具 → 更新历史 4. 返回最终结果

**不做什么：** - 不处理 HTTP（那是 ChatController 的事） - 不发 HTTP 请求（那是 AnthropicClient 的事） - 不执行工具逻辑（那是 Tool 的事）

**AnthropicClient（LLM API 调用）**

| 属性     | 值                          |
|--------|----------------------------|
| 类标记    | @Component                 |
| 生命周期   | 单例                         |
| 依赖     | AgentProperties (配置)       |
| 线程模型   | 同步阻塞 (调 HTTP 时当前线程等待)      |
| C++ 类比 | Thrift Client 桩 (发 RPC 请求) |

**做什么：** 1. 把 List<Message> 转成 Claude API 需要的 JSON 格式 2. 用 Java 内置 HttpClient 发 HTTP POST 请求 3. 解析 Claude API 返回的 JSON 响应 4. 提取文本 (text) 和工具调用 (tool\_use) 5. 返回 AnthropicResponse 对象

**不做什么：** - 不管理对话历史 - 不执行工具 - 不控制循环

**ToolRegistry (工具管理)**

| 属性     | 值                                      |
|--------|----------------------------------------|
| 类标记    | @Component                             |
| 生命周期   | 单例                                     |
| 依赖     | 所有实现了 AgentTool 接口的类的实例                |
| 线程模型   | 同步执行 (工具执行在当前线程)                       |
| C++ 类比 | std::map<std::string, HandlerFunction> |

**做什么：** 1. 构造函数接收所有 AgentTool 实例 (Spring 自动注入) 2. 把 List 转成 Map (key=工具名, value=工具实例) 3. 提供 execute(name, input) 方法, 按名字查找并执行工具

**不做什么：** - 不定义工具逻辑 - 不控制循环 - 不发 HTTP 请求

**AgentTool (工具接口)**

| 属性     | 值                                                                      |
|--------|------------------------------------------------------------------------|
| 类型     | interface                                                              |
| C++ 类比 | 纯虚基类 <code>class AgentTool { virtual string execute(...) = 0; }</code> |

定义了三个方法： - `getName()` — 工具名称 - `getDescription()` — 工具描述 - `execute(Map<String, Object> input)` — 执行工具

为什么用接口： - 新增工具只需新建类实现这个接口 + 加 `@Component` - 不需要修改 `ToolRegistry` 或 `AgentService` - Spring 自动发现新工具并注入

FileReadTool / FileWriteTool / BashTool (具体工具)

| 属性     | 值                       |
|--------|-------------------------|
| 类标记    | <code>@Component</code> |
| 生命周期   | 单例                      |
| C++ 类比 | 具体的 handler 函数实现        |

每个工具实现三个方法： - 返回名字 (如 "file\_read") - 返回描述 (告诉 Claude 这个工具做什么) - 执行具体逻辑 (读文件、写文件、执行命令)

2.3 对象创建和连接的全过程

### Spring Boot 启动:

1. 扫描到 `@Component public class FileReadTool`
  - Spring 调用 `new FileReadTool()`
  - 实例存入容器 (key="fileReadTool", value=实例)
2. 扫描到 `@Component public class FileWriteTool`
  - Spring 调用 `new FileWriteTool()`
  - 实例存入容器 (key="fileWriteTool", value=实例)
3. 扫描到 `@Component public class BashTool`
  - Spring 调用 `new BashTool()`
  - 实例存入容器 (key="bashTool", value=实例)
4. 扫描到 `@Component public class ToolRegistry`
  - Spring 看到构造函数参数是 `List<AgentTool>`
  - Spring 在容器里找到所有 `AgentTool` 实现 (`FileReadTool`、`FileWriteTool`、`BashTool`)
  - Spring 把它们放到 `List` 里
  - Spring 调用 `new ToolRegistry(listOfThreeTools)`
  - 实例存入容器
5. 扫描到 `@Component public class AnthropicClient`
  - Spring 看到 `@Autowired private AgentProperties`
  - Spring 在容器里找到 `AgentProperties` 实例
  - Spring 调用 `new AnthropicClient()`
  - Spring 把 `AgentProperties` 实例赋值给 `anthropicClient.properties`
  - 实例存入容器
6. 扫描到 `@Service public class AgentService`
  - Spring 看到两个 `@Autowired` 字段
  - Spring 在容器里找到 `AnthropicClient`、`ToolRegistry` 实例
  - Spring 调用 `new AgentService()`
  - Spring 把 `AnthropicClient`、`ToolRegistry` 实例赋值给对应字段
  - 实例存入容器
7. 扫描到 `@RestController public class ChatController`
  - Spring 看到 `@Autowired private AgentService`
  - Spring 在容器里找到 `AgentService` 实例
  - Spring 调用 `new ChatController()`
  - Spring 把 `AgentService` 实例赋值给 `chatController.agentService`
  - 实例存入容器

## 三、进程/线程维度：谁在什么线程上做什么

### 3.1 进程

操作系统只看到一个进程：`java.exe`

这个进程内部：

└─ JVM

└─ 堆内存（所有 **Java** 对象都在这里）

└─ 方法区（类的元数据、静态变量）

└─ 栈内存（每个线程有自己的栈）

└─ 线程

### 3.2 线程模型全景

```

java.exe 进程
├── [主线程] main() 启动 Spring Boot
│   ├── 创建 Spring 容器
│   ├── 创建所有 @Component 对象
│   ├── 处理 @Autowired 依赖
│   └── 启动 Tomcat → 主线程结束
├── [Tomcat 线程池] 处理 HTTP 请求
│   ├── 时刻 T1: 用户 A 发 POST /api/chat {message: "hello"}
│   │   ├── [Tomcat 线程 1] ChatController.chat()
│   │   │   ├── request.message() → "hello"
│   │   │   ├── agentService.run("hello")
│   │   │   │   └── 返回 CompletableFuture (立即返回! 不阻塞)
│   │   │   └── 返回 CompletableFuture → [Tomcat 线程 1 释放]
│   │   └── [ @Async 线程 A ] AgentService.run("hello")
│   │       ├── 第 1 轮: anthropicClient.chat(history)
│   │       │   └── [线程 A 被阻塞, 等待 HTTP 响应]
│   │       ├── 第 2 轮: claude 返回 text, 无工具调用
│   │       └── CompletableFuture.complete(result)
│   │           └── Tomcat 收到完成通知, 返回响应给用户 A
│   ├── 时刻 T2: 用户 B 发 POST /api/chat {message: "world"}
│   │   ├── [Tomcat 线程 2] ChatController.chat()
│   │   │   └── ... 同上
│   │   └── [ @Async 线程 B ] AgentService.run("world")
│   │       └── ... 同上
│   └── 线程 A 和线程 B 并发执行, 互不阻塞
├── [ @Async 线程池 ] 执行 @Async 标记的方法
│   ├── 线程 A: 正在跑 AgentService.run()
│   ├── 线程 B: 正在跑 AgentService.run()
│   └── ...
└── [GC 线程] 后台垃圾回收 (自动, 开发者不需要关心)

```

### 3.3 一个完整请求的线程视角

时间轴（从上到下）：

T0: Tomcat 线程 3 空闲，等待请求

T1: 用户发 POST /api/chat

- [Tomcat 线程 3] 开始工作
  - ChatController.chat(request)
    - agentService.run("帮我读 main.java")
    - 返回 CompletableFuture
- [Tomcat 线程 3] 完成，立刻释放，可以处理下一个请求

T2: [@Async 线程 C] 开始工作

- AgentService.run("帮我读 main.java")
  - 创建 history = [Message("user", "帮我读 main.java")]

第 1 轮：

- anthropicClient.chat(history)
  - 构建 JSON 请求体
  - 发 HTTP POST 到 Claude API
  - [@Async 线程 C 阻塞等待 HTTP 响应] ← 线程在这里等待
  - 收到 JSON 响应
  - 解析: tool\_use(name="file\_read", input={path: "main.java"})
- toolCalls 不为空
- toolRegistry.execute("file\_read", {path: "main.java"})
  - FileReadTool.execute()
  - Files.readString(...) // 读文件，可能阻塞在磁盘 I/O
- 工具结果加入 history

第 2 轮：

- anthropicClient.chat(history)
  - 发 HTTP POST
  - [@Async 线程 C 再次阻塞等待 HTTP 响应]
  - 收到 JSON 响应
  - 解析: 纯文本，无 tool\_use
- toolCalls 为空
- 循环结束

- CompletableFuture.complete(result)
- [@Async 线程 C 释放]

T3: Tomcat 检测到 CompletableFuture 完成

- 把结果转成 JSON 返回给用户

### 3.4 和不加 @Async 的区别

如果不加 @Async (同步模式) :

T1: 用户发请求

- [Tomcat 线程 3] ChatController.chat()
- agentService.run("帮我读 main.java") ← 同步调用, 阻塞在这里
  - 调 Claude API (等待 2 秒)
  - 执行工具 (等待 0.1 秒)
  - 调 Claude API (等待 2 秒)
  - 返回结果 (共等待 4.1 秒)
- 返回响应
- [Tomcat 线程 3] 释放 (共占用 4.1 秒)

T2: 用户 B 请求 (在 T1 之后 1 秒)

- 没有空闲 Tomcat 线程! (线程 3 还被阻塞着)
- 必须等到 T1 完成 (再等 3.1 秒) 才能被处理

加了 @Async (异步模式) :

T1: 用户发请求

- [Tomcat 线程 3] ChatController.chat()
- agentService.run() ← 立即返回 CompletableFuture
- [Tomcat 线程 3] 释放 (只占用了几毫秒)

→ [@Async 线程 C] 开始跑 Agent 循环 (不占用 Tomcat 线程)

T2: 用户 B 请求 (在 T1 之后 1 秒)

- [Tomcat 线程 4] 立刻处理 (因为线程 3 早已释放)

3.5 线程安全分析

| 共享数据                     | 有线程安全问题吗 | 原因                                               |
|--------------------------|----------|--------------------------------------------------|
| ToolRegistry.tools (Map) | 无        | 只在构造函数里写入一次, 之后只读                                |
| AnthropicClient 的字段      | 无        | HttpClient.send() 是线程安全的                         |
| AgentService 的字段         | 无        | 每个 @Async 线程创建自己的 List<Message> history (方法局部变量) |
| FileReadTool 等工具的字段      | 无        | 工具类无状态 (只有方法, 没有可变的成员变量)                         |

### 为什么没有线程安全问题：

- Spring 默认是单例模式（每个类只创建一个实例）
- 但所有共享的实例都是**无状态的**（不保存请求相关的数据）
- 每个请求的数据（对话历史）是方法局部变量，每个线程有自己的栈，互不干扰

### 类比 C++：

```
// C++ 多线程 RPC Server
class RpcServer {
    // 共享的 handler 注册表（只读，线程安全）
    std::map<std::string, HandlerFunc> handlers_;

    void handle_request(Request req) {
        // 每个线程有自己的局部变量，线程安全
        std::vector<std::string> local_history = {};
        // ... 处理请求
    }
};
```

## 四、三个维度的关系总结



## 五、C++ 和 Java/Spring Boot 最根本的差异：线程谁管

### 5.1 你在 C++ 里的做法

在 C++ 后端服务里，你习惯这样：

```

// 每个组件启动自己的工作线程
class RpcServer {
    std::thread worker_thread_;

    void start() {
        worker_thread_ = std::thread([this] {
            while (running_) {
                Request req = receive();    // 阻塞接收请求
                Response resp = handle(req);
                send(resp);
            }
        });
    }
};

class AgentLoop {
    std::thread loop_thread_;

    void start() {
        loop_thread_ = std::thread([this] {
            while (running_) {
                Response resp = llm.chat(history);
                if (resp.hasToolCall()) {
                    executeTool(resp.toolCall);
                }
            }
        });
    }
};

// main() 里手动启动所有组件
int main() {
    RpcServer server;
    server.start();    // 手动启动线程

    AgentLoop agent;
    agent.start();    // 手动启动线程

    server.worker_thread_.join();    // 手动等待线程结束
    agent.loop_thread_.join();
}

```

**C++ 的特点：** - 每个组件有自己的线程，线程的生命周期你自己管理 - 你决定什么时候 `start()`、什么时候 `join()` - 线程里通常是一个 `while(running)` 循环，自己取任务、自己处理 - 你控制一切——"这个组件分配多少线程、优先级是多少、怎么同步"

## 5.2 Java/Spring Boot 的做法

```
// 你没有写任何线程创建代码
// 没有 std::thread, 没有 start(), 没有 join()

@RestController
public class ChatController {
    // 只是一些字段和方法, 没有线程
    @PostMapping("/chat")
    public Map<String, String> chat(@RequestBody ChatRequest request) {
        return Map.of("reply", agentService.run(request.message()));
    }
}

@Service
public class AgentService {
    @Async // ← 就这一个注解, Spring 会把这个方法放到线程池里执行
    public CompletableFuture<String> run(String userMessage) {
        // 循环逻辑
    }
}
```

**Java/Spring Boot 的特点:** - 你不需要创建线程, 框架帮你创建和管理线程池 - 你只需要写业务逻辑方法, 框架决定在哪个线程上执行它 - Tomcat 线程池: Spring 把 HTTP 请求 dispatch 到线程池里的某个线程上执行你的 Controller 方法 - @Async 线程池: Spring 把标记了 @Async 的方法放到线程池里异步执行 - 线程是"被动"的——它们在线程池里闲着, 等 Spring 分配任务

## 5.3 对比

**C++: 线程主动模式**

每个组件 = 一个独立线程 + `while(true)` 循环

RPC线程

```
while(1){
  recv();
  handle();
  send();
}
```

Agent线程

```
while(1){
  chat();
  loop();
}
```

Worker线程

```
while(1){
  exec();
  report();
}
```

你控制：创建多少线程、什么时候启动、怎么同步

**Java/Spring Boot: 线程被动模式**

框架维护线程池，你的代码只是"被调用"

Tomcat 线程池 (N 个线程空闲等待)

线程1: 闲着 → 来请求了 → `ChatController.chat()` → 闲着  
 线程2: 闲着 → 来请求了 → `ChatController.chat()` → 闲着  
 线程3: 闲着...

@Async 线程池 (N 个线程空闲等待)

线程A: 闲着 → 有任务 → `AgentService.run()` → 闲着  
 线程B: 闲着...

框架控制：创建多少线程、什么时候用哪个线程

## 5.4 这意味着什么

**你不需要做的事情 (框架帮你做了)：** - 创建线程 - 管理线程生命周期 (启动、停止、回收) - 分配任务到线程 - 线程间的同步 (锁、条件变量)

**你只需要做的事情：** - 写业务方法 (`chat()`、`run()`、`execute()` 等) - 加注解告诉框架这个方法的特性 (@Async 异步执行、@PostMapping 由 HTTP 触发) - 框架负责把方法调用

分配到合适的线程

**这和 C++ 的思维完全相反：**

**C++：** 先想"我需要几个线程"，再想"每个线程干什么"

**Java：** 先想"我需要什么方法"，再想"这个方法该同步还是异步执行"

**为什么 Java 这样设计：** - Java 在服务器端用得最多（处理 HTTP 请求） - 服务器场景的特点是：大量短任务、I/O 等待多、并发高 - 线程池 + 任务 dispatch 的模式最适合这种场景 - 让开发者专注于业务逻辑，不用操心线程管理

**你之前用 C++ 做后端 RPC 服务时，也接触过类似的概念：** - Thrift 的 TThreadPoolServer 就是预先创建好线程池，收到 RPC 请求后分配线程执行 - 你的 handler 代码不需要自己创建线程，Thrift 帮你管理 - Spring Boot 做的是同样的事，只是管理范围更广（HTTP + @Async + 定时任务等）

## 5.5 再深一层：Spring 是怎么做到的

```

// 你写了这些:
@PostMapping("/chat")
public void chat(Request req) { ... }

@Async
public void run(String msg) { ... }

// Spring 内部做了什么:
//
// 对于 @PostMapping:
//   Spring 扫描到 chat() 方法
//   → 注册路由: POST /api/chat → 调用这个方法
//   → 当请求到来时:
//       从 Tomcat 线程池取出一个空闲线程
//       在那个线程上调用 chat(req)
//       方法返回后, 线程归还给线程池
//
// 对于 @Async:
//   Spring 扫描到 run() 方法
//   → 生成一个代理对象
//   → 当代码调用 run(msg) 时:
//       代理截获调用
//       把 run(msg) 包装成一个任务
//       把任务放进 @Async 线程池的队列
//       线程池的某个线程取出任务并执行
//       调用方立即得到 CompletableFuture

```

**换句话说：**你写的每一个 `@PostMapping` 方法和 `@Async` 方法，都是被框架**调度执行的**，不是在你自己创建、自己管理的线程上跑的。